

METHODOLOGY

System Architecture Overview

The platform is a decentralised application in which patients control who can read their medical records. Its layered architecture is shown in Figure 1. Three user roles (administrator, patient, and doctor) interact through a React front-end, which connects to the MetaMask wallet. MetaMask signs and submits transactions to a local Ethereum network running on Ganache, where the MedicalRecord smart contract is deployed and enforces all access rules. Write actions such as adding a record or granting access change state, cost gas, and require MetaMask confirmation, while read actions such as viewing a record are free view calls that return data directly to the interface.

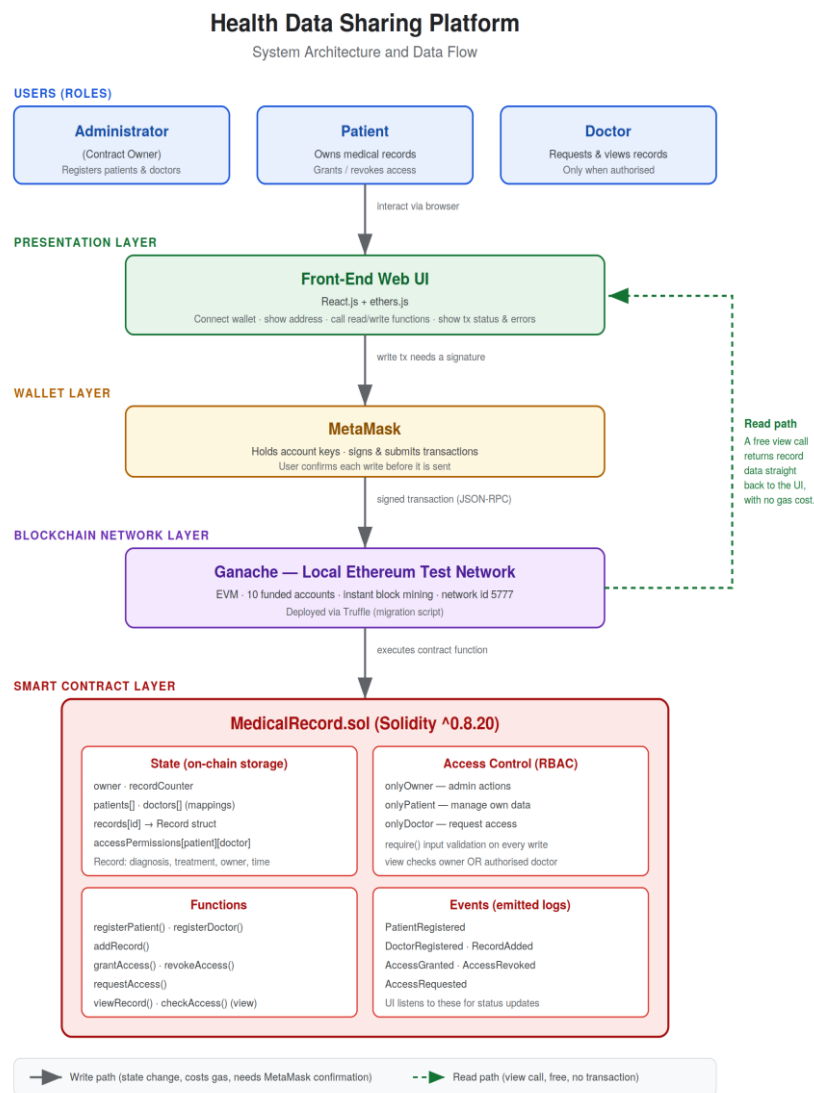


Figure 1. System architecture and data flow of the platform.

Smart Contract Design

The system is built around a single Solidity contract, MedicalRecord (version 0.8.20). It stores the owner address, a record counter, two mappings that track registered patients and doctors, a nested mapping (accessPermissions) recording which doctor each patient has authorised, and a mapping of record identifiers to a Record struct holding the diagnosis, treatment, owning patient, and timestamp. Functions are separated by responsibility: the administrator registers users (registerPatient, registerDoctor); patients add records (addRecord) and manage access (grantAccess, revokeAccess);

doctors request access (`requestAccess`); and two view functions (`viewRecord`, `checkAccess`) read data. Six events are emitted so the front-end can track each action.

Access Control and Security

Access control is enforced through role-based modifiers: `onlyOwner` for administrative functions, `onlyPatient` for data management, and `onlyDoctor` for access requests. The `viewRecord` function additionally permits a record to be read only by its owning patient or an authorised doctor. The contract relies on Solidity 0.8's built-in overflow protection instead of an external library, and validates every input with `require` statements carrying descriptive error messages, for example rejecting the zero address or empty record fields.

Gas Optimisation

Several choices reduce gas cost: the read functions are marked `view` and create no transactions; the string parameters of `addRecord` use `calldata` to avoid an extra copy; events are used for off-chain logging instead of on-chain storage; and the Solidity optimiser was enabled during compilation.

Technologies and Justification

Ethereum was selected as a mature, widely supported smart-contract platform with standard tooling, and Solidity 0.8 for its built-in arithmetic safety. Development used Truffle and Ganache, the stack covered in the laboratory sessions: Truffle compiles, deploys, and runs the Mocha and Chai test suite, while Ganache provides a local network with ten funded accounts and instant mining for fast, cost-free testing. MetaMask handles wallet signing, and the React front-end interacts with the contract through `ethers.js`. A local Ganache network was chosen over a public testnet such as Sepolia to avoid faucet dependence and public block latency during development; a Sepolia deployment is identified as future work.

Deployment

The contract was deployed to a local Ganache network using a Truffle migration script. Compilation targeted the Paris EVM version, because the default Shanghai target emits the `PUSH0` opcode that the local Ganache build does not support. Truffle records the contract address and ABI, which are then supplied to the front-end.

RESULTS AND ANALYSIS

Deployment and Functional Testing

The contract compiled without errors or warnings and deployed successfully to a local Ganache test network, using approximately 1,041,496 gas. It was then validated with an automated suite of twenty-nine tests written in Mocha and Chai, all of which passed. The suite covers four categories: the happy path (each function with valid input and the correct event), access control (role modifiers rejecting unauthorised callers), edge cases (zero address, empty fields, duplicate registration, and non-existent records), and multi-account interaction using five accounts. Because every function and every `require` condition is exercised, the suite achieves full coverage of the contract's functions and revert paths. The full test-case table and deployment evidence are provided with the development artefact.

Gas Consumption

Gas was measured for the deployment and each state-changing function, as shown in Table 1. The two registration functions cost a similar amount; `requestAccess` and `revokeAccess` are the cheapest; and `addRecord` is the most expensive at 161,011 gas because it writes a struct containing two strings into persistent storage. This confirms that on-chain string storage is the main driver of cost in the contract.

Operation	Gas Used
Deployment (constructor)	1,041,496
registerPatient	47,498
registerDoctor	47,520
addRecord	161,011
grantAccess	50,008
requestAccess	27,708
revokeAccess	28,056

Table 1. Gas consumption per operation.

Analysis of Findings

The results show that the contract meets its objective of letting patients control access to their data. The passing access-control tests confirm that the permission model works correctly, and the gas measurements show that it operates at a predictable cost. The gap between the inexpensive permission functions and the more costly addRecord function indicates where future optimisation, such as moving large data off-chain, would be most useful.